UNIVERSAL TECHNOLOGY AND MANAGEMENT UNIVERSITY

UTAMU

For an Open Mind

AN AI-DRIVEN SOFTWARE SYSTEM FOR DETECTING PARAPHRASED AND AI-GENERATED ACADEMIC TEXT USING SENTENCE-BERT AND STYLOMETRIC ANALYSIS

By

WEMBABAZI RICH	SEP24/BIST/4585U/H
BYAMUKAMA JOHNBOSCO	SEP24/BIST/4749U/F

DEPARTMENT OF INFORMATION SYSTEMS AND TECHNOLOGY SCHOOL OF COMPUNTING AND ENGINEERING

A Concept Paper submitted to the School of Computing and Engineering For the Study Leading to a project Proposal in Partial Fullfillmet of Requirements for the Award of the Degree of Bachelor of Science in Information Systems And Technology Of Universal Technology and Management University.

Supervisor : Mukama Alex

Department of Information Systems and Technology

School of Computing and Engineering, Uganda Technology and Management University
amukama@utamu.ac.ug, +256777328921

September 2026

Table of Contents

AI-DRIVEN ACADEMIC INTEGRITY SYSTEM: A PURELY SOFTWARE-BASED ARCHITECTURE INTEGRATING SENTENCE-BERT VECTOR EMBEDDINGS AND STATISTICAL STYLOMETRIC PROFILING FOR PARAPHRASE AND AI-GENERATED TEXT DETECTION 1

ABBREVIATIONS 3

1. Introduction 4

2. Background to the Problem 4

3. Problem Statement 5

4. Objectives 5

4.1) Main Objective 5

4.2) Specific Objectives 5

3. Methodology 6

3.1) Phase 1: Document Ingestion and Sentence-Segmentation Pipeline 6

3.2) Phase 2: Semantic Vector Analysis Module 6

3.3) Phase 3: Statistical Stylometric Engine 7

3.4) Phase 4: Interactive Web Interface and Color-Coded Heatmap 8

4. Outcomes 8

5. References 9

ABBREVIATIONS

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **BERT:** Bidirectional Encoder Representations from Transformers
- **DOCX:** Microsoft Word XML Document Format
- **LLM:** Large Language Model
- **NLP:** Natural Language Processing
- **NLTK:** Natural Language Toolkit
- **PDF:** Portable Document Format
- **REST:** Representational State Transfer
- **SBERT:** Sentence-BERT
- **SDLC:** Software Development Life Cycle
- **UI / UX:** User Interface / User Experience
- **UTF-8:** 8-bit Unicode Transformation Format

1. Introduction

The rapid expansion of artificial intelligence tools and automated rephrasing software has fundamentally altered academic writing, rendering traditional N-gram and string-matching plagiarism detectors largely ineffective against modern techniques like synonym substitution and AI-driven paraphrasing (Najjar et al., 2025). To address these critical vulnerabilities, this project introduces a purely software-based academic integrity platform that replaces rigid word-for-word string matching with a dual-engine analytical framework. By integrating Sentence-BERT (SBERT) vector embeddings to measure deep semantic similarity via Cosine Distance (Reimers & Gurevych, 2019; Tang et al., 2025) alongside Perplexity and Burstiness profiling to evaluate statistical token predictability and structural variance, the system accurately detects both contextually rephrased plagiarism and machine-generated text in a lightweight, locally deployable application.

2. Background to the Problem

The rapid proliferation of artificial intelligence tools and automated text-rephrasing software has fundamentally altered how students compose academic papers, presenting a severe challenge to traditional academic integrity frameworks (Najjar et al., 2025). Historically, universities have relied on legacy software utilizing N-gram and string-matching algorithms to scan databases for verbatim word overlap. However, modern students can effortlessly bypass these systems using tools like ChatGPT and QuillBot to swap synonyms, restructure sentences, and alter grammatical voice while retaining the exact original context (Najjar et al., 2025). As a result, string-matching detectors yield false-negative 0% similarity scores for heavily paraphrased or AI-assisted submissions, failing to identify stolen ideas or uncredited machine authorship (Najjar et al., 2025). Furthermore, existing commercial detection platforms often operate as proprietary cloud services that require costly subscriptions, demand heavy infrastructure, and present data privacy risks by uploading student intellectual property to third-party databases (Najjar et al., 2025).

To bridge this gap, a clear need exists for a lightweight, purely software-based solution capable of evaluating semantic meaning alongside statistical writing patterns without relying on external web-crawling infrastructure. Deep contextual models like Sentence-BERT (SBERT) project full sentences into high-dimensional vector spaces, allowing software to measure concept proximity via Cosine Similarity regardless of vocabulary alterations or structural rephrasing (Reimers & Gurevych, 2019; Tang et al., 2025).

Simultaneously, statistical metrics such as Perplexity and Burstiness can analyze token predictability and sentence-length variations to flag text generated directly by large language models. Combining SBERT semantic vector modeling with Perplexity and Burstiness profiling into a unified, locally deployable application delivers an effective, dual-engine academic integrity system that reliably catches paraphrase plagiarism and AI authorship.

3. Problem Statement

Traditional plagiarism tools rely on lexical string-matching algorithms, consistently failing to detect AI-assisted paraphrasing and synonym swapping. Modern generative AI and rephrasing software allow students to alter text structures while retaining core meanings, leading legacy checkers to return false-negative 0% similarity scores. To solve these gaps, this project builds a lightweight, local software application combining Sentence-BERT vector embeddings for semantic similarity with Perplexity and Burstiness profiling for AI authorship detection.

4. Objectives

4.1) Main Objective

To design, develop, and evaluate a purely software-based academic integrity platform that integrates Sentence-BERT (SBERT) semantic vector analysis and statistical stylometric profiling (Perplexity and Burstiness) to detect paraphrase plagiarism and AI-generated text in academic submissions.

4.2) Specific Objectives

i. **Requirements Analysis:** To identify and analyse the functional and non-functional requirements of a plagiarism detection system capable of detecting paraphrased content and identifying characteristics associated with AI-generated text in PDF and DOCX documents.

ii. **System Design:** To design the system architecture, document-processing workflow, semantic similarity analysis module, stylometric analysis engine, and user interface for presenting plagiarism detection and AI-content analysis results.

iii. **System Development and Implementation:** To develop a document ingestion and sentence-segmentation pipeline using PyPDF2, python-docx, and NLTK; implement semantic similarity analysis using Sentence-BERT (all-MiniLM-L6-v2) and cosine

similarity; develop a stylometric engine using perplexity and burstiness measures; and create a React.js interface with drag-and-drop uploads, adjustable similarity thresholds, and colour-coded sentence highlighting.

iv. **System Testing and Evaluation:** To test and evaluate the system's accuracy, reliability, and performance in detecting paraphrased content and identifying characteristics associated with AI-generated text using sample documents, while identifying and correcting errors to improve the system's effectiveness.

5. Methodology

This project adopts an iterative **Agile Software Development Life Cycle (SDLC)** model. The core development work is divided into four distinct phases, mapping directly to the project's four specific objectives:

5.1) Phase 1: Document Ingestion and Sentence-Segmentation Pipeline

Corresponding to Specific Objective 1: Developing the ingestion and segmentation pipeline.

5.1.1) **File Parsing & Text Extraction:** Implement custom Python modules using PyPDF2 (for PDF documents) and python-docx (for Word files) to extract unformatted raw text streams from uploaded academic files. (Python Software Foundation, 2023)

5.1.2) **Text Normalization:** Strip non-printable characters, clean excess whitespace, and preserve basic paragraph structures to ensure high downstream tokenization accuracy.

5.1.3) **Sentence Segmentation:** Utilize NLTK.sent_tokenize to split cleaned document text into discrete sentence-level units $S = \{s_1, s_2, \dots, s_n\}$. These individual sentences serve as the baseline input units for vector transformation and highlight mapping.

5.2) Phase 2: Semantic Vector Analysis Module

Corresponding to Specific Objective 2: Implementing SBERT embeddings and Cosine Similarity.

5.2.1) **Vector Encoding:** Integrate the pre-trained sentence-transformers/all-MiniLM-L6-v2 Sentence-BERT model to convert each extracted sentence s_i into a dense 384-dimensional vector embedding $\vec{v}_i \in \mathbb{R}^{384}$.

5.2.2) **Pairwise Similarity Calculation:** Evaluate the mathematical proximity between sentence vectors from Document A ($\vec{A}$) and Document B ($\vec{B}$) using Cosine Distance:

$$\text{Cosine Similarity}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\|\|\vec{B}\|}$$

5.2.3) **Paraphrase Identification:** Map cross-document sentence matches exceeding a similarity threshold (≥ 0.75), capturing synonym substitutions, active-to-passive voice changes, and structural rephrasing that bypass legacy string-matching tools.

5.3) Phase 3: Statistical Stylometric Engine

Objective 3: Building the Perplexity and Burstiness AI-detection engine.

5.3.1) **Perplexity Calculation:** Measure token predictability across a sentence sequence $W = (w_1, w_2, \ldots, w_N)$ using a localized causal language model (Jurafsky & Martin, 2023):

$$\text{Perplexity}(W) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P\left(w_i \mid w_1, w_2, \ldots, w_{i-1}\right)\right)$$

Lower perplexity scores indicate highly predictable token choices characteristic of machine-generated text (Gehrmann et al., 2019; Mitchell et al., 2023).

5.3.2) **Burstiness Calculation:** Evaluate structural variance across sentence lengths $L = \{l_1, l_2, \ldots, l_k\}$ to measure writing rhythm.

$$\text{Burstiness} = \frac{\sigma_L - \mu_L}{\sigma_L + \mu_L}$$

Human writing naturally exhibits high variation in sentence lengths (higher burstiness), while generative AI models produce uniform sentence lengths (lower burstiness) (Mitchell et al., 2023; Tang et al., 2025).

5.3.3) **Aggregate AI Score:** Combine normalized Perplexity and Burstiness metrics to compute an overall document AI Probability Score (0% to 100%) (Gehrmann et al., 2020).

5.4) Phase 4: Interactive Web Interface and Color-Coded Heatmap

- **Frontend Dashboard:** Build an interactive single-page React.js web dashboard styled with Tailwind CSS, featuring drag-and-drop file upload fields for dual-document evaluation (Meta Open Source, 2023).
- **Threshold Controls:** Include interactive slider controls that allow academic evaluators to adjust semantic similarity and AI sensitivity thresholds dynamically (Shneiderman, 2003).
- **Side-by-Side Heatmap Renderer:** Dynamically highlight uploaded sentences on screen based on engine calculations:
    - **Red Highlight:** High semantic/vector similarity match ($\geq 85\%$).
    - **Yellow Highlight:** Moderate rephrased or paraphrased match ($70\% - 84\%$).
    - **Blue/Purple Highlight:** Low perplexity and burstiness region flagged as AI-generated.
    - **Unflagged Text:** Original human-written text.

6. Outcomes

- **Document Processing Pipeline:** Automated Python module using PyPDF2, python-docx, and NLTK to extract and segment .pdf and .docx text into sentence-level units.
- **Semantic Paraphrase Detector:** Local SBERT (all-MiniLM-L6-v2) engine converting sentences into 384-D vector embeddings to identify paraphrased text via Cosine Similarity (≥ 0.75).
- **Stylometric AI Detector:** Statistical engine measuring token Perplexity and sentence-length Burstiness to output an overall AI Probability Score ($0\% - 100\%$).

- **Interactive React.js Dashboard:** Web UI built with React.js and Tailwind CSS featuring drag-and-drop file uploads, threshold sliders, and a side-by-side color-coded sentence heatmap.

- **Offline Deployment Package:** Docker container / Python venv setup ensuring 100% local execution without external API dependencies.

- **Performance Targets:** $\geq$ 85% paraphrase accuracy, $\geq$ 90% AI precision, $\geq$ 0.85 F_1-score, and $<$ 3.0 seconds/page latency.

7. References

Bird, S. L. (2023). *Natural Language Processing with Python: Analyzing text with the Natural Language Toolkit (2nd ed.).* O'Reilly Media.

Foundation, P. S. (2023). Python language reference (Version 3.11) [Computer software]. From https://www.python.org/

Jurafsky, D. &. (2023). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition* (3rd ed. draft ed.). Stanford University Press. . From https://web.stanford.edu/~jurafsky/slp3/

Mitchell, E. L. (2023). DetectGPT: Zero-shot machine-generated text detection using probability curvature. *Proceedings of the 40th International Conference on Machine Learning (ICML), 202, 24950–24962*. doi:https://proceedings.mlr.press/v202/mitchell23a.html

OpenAI. (2023). GPT-4 technical report (arXiv:2303.08774). *arXiv.* From https://doi.org/10.48550/arXiv.2303.08774

Reimers, N. &. (2023). Sentence-transformers: Fast sentence embeddings using Sentence-BERT models (all-MiniLM-L6-v2 documentation). *Hugging Face Transformers Ecosystem.* From https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Source, M. O. (2023). *React: A JavaScript library for building user interfaces (Version 18.2) [Computer software].* Meta Platforms. From https://react.dev/